Defining Object-Oriented Programming

(https://github.com/learn-co-curriculum/python-p3-defining-object-oriented-programming) (https://github.com/learn-co-curriculum/pytho
p3-defining-object-oriented-programming/issues/ne

Learning Goals

- Define Object Oriented Programming
- Explain the benefits of Object Oriented Programming

Key Vocab

- **Class**: a bundle of data and functionality. Can be copied and modified to accomplish a wide variety of programming tasks.
- **Object**: the more common name for an instance. The two can usually be used interchangeably.
- **Object-Oriented Programming**: programming that is oriented around data (made mobile and changeable in **objects**) rather than functionality. Python is an object-oriented programming language.
- **Function**: a series of steps that create, transform, and move data.
- **Method**: a function that is defined inside of a class.

Introduction

Object-Oriented Programming (OOP)

*An object-oriented approach to application development makes programs more intuitive to design, faster to develop, more amenable to modification, and easier to understand. —**Object-Oriented Programming with Objective-C** (https://developer.apple.com/library/ios/documentation/Cocoa/Conceptual/OOP_ObjC/Introduction/Introduction.html#//apple_ref/doc/uid/TP40005147-CH1-SW2), Apple Inc.*

It's natural to wonder, "how can a string of ones and zeroes be referred to as an 'object'?" The use of the word "object" is an abstraction of thought. An "object" in code has no more physical form than does a word in any human language. Sure, words have physical representations: speaking a word causes air to vibrate in a sound wave, ink on a page can be shaped into symbols that represent the word, a meaning can be pointed at or mimed out; but none of these are the word itself. Human language is a system of abstraction: it communicates the *idea* of a thing, but not the thing itself.

Translation: "This is not a pipe." - ***The Treachery of Images*** (https://en.wikipedia.org/wiki/The_Treachery_of_Images) , **René Magritte** (https://en.wikipedia.org/wiki/Ren%C3%A9_Magritte) , 1927

This image of a pipe is no more a pipe than the word "pipe" is a pipe; in the same way, a code object named `pipe` is not a pipe, but only another form of representing a pipe.

> As humans, we're constantly faced with myriad facts and impressions that we must make sense of. To do so, we must abstract underlying structure away from surface details and discover the fundamental relations at work. Abstractions reveal causes and effects, expose patterns and frameworks, and separate what's important from what's not. Object orientation provides an abstraction of the data on which you operate moreover, it provides a concrete grouping between the data and the operations you can perform with the data—in effect giving the data behavior.
> —***Object-Oriented Programming with Objective-C***

> (https://developer.apple.com/library/ios/documentation/Cocoa/Conceptual/OOP_ObjC/Articles/ooOOP.html#//apple_ref/doc/uid/TP40005149-CH8-SW3) , Apple Inc.

A code object representing a water pipe (instead of a smoking pipe) might contain values for `length` , `diameter` , `material` , and `manufacturer` . The bundling of these individual pieces of information together begins to form a larger whole.

Object-Oriented Programming, however, does more than just bundle up individual pieces of data that represent a "thing" — it also bundles customized functions that can be performed *on* that data. These are called **methods**: behaviors that an object performs upon its internal data and even upon other code objects.

An object in code is a thing with all the data and all the logic required to complete a task. Objects are models and metaphors for the problems v solve in code.

Object-oriented programming was born from the trend of making digital lives reflect our real lives. In the 1970's, **Adele Goldberg** (https://en.wikipedia.org/wiki/Adele_Goldberg_%28computer_scientist%29) and **Alan Kay** (https://en.wikipedia.org/wiki/Alan_Kay) developed a object-oriented language at Xerox PARC called Smalltalk, which was used in the first personal computer.

Python comes with a few types of Objects to get us started, things like `int` , `str` , `list` , etc. We call these base types of Objects "Primitives. But what if we wanted to create a new type in our programming universe, a new kind of object for our code? That's what the `class` keyword a object orientation allows us to do.

Resources

- **Object-Oriented Programming (OOP) in Python 3** (https://realpython.com/python3-object-oriented-programming/)
- **Object-Oriented Programming with Objective-C** (https://developer.apple.com/library/ios/documentation/Cocoa/Conceptual/OOP_ObjC/Introduction/Introduction.html#//apple_ref/doc/uid/TP40005149-CH1-SW2)
- **Smalltalk - Wikipedia** (https://en.wikipedia.org/wiki/Smalltalk)
- **Differences Between Procedural and Object-Oriented Programming** (https://www.geeksforgeeks.org/differences-between-procedural-and-object-oriented-programming/#:%7E:text=Object%2Doriented%20programming%20is%20based,the%20concept%20of%20procedure%20abstraction.)